

Tutorial - Semana 4, Encontro 2

Introdução

No Encontro 1, o projeto ganhou seu primeiro Autoload: o `GameManager`, construído como **Orchestration** e registrado em Project Settings. Este encontro completa o Módulo 2 desta semana em duas frentes. Primeiro, uma discussão conceitual breve sobre onde vive o input de alto nível do jogador no Godot, sem separação nativa entre "quem controla" e "o que é controlado". Depois, a construção do segundo Autoload da disciplina, o `SaveManager`, responsável por manter dados persistentes entre trocas de cena — o primeiro passo em direção ao save/load completo que será construído na Semana 7.

Como no Encontro 1, o `SaveManager` é construído como **Orchestration**. Cada trecho de GDScript no tutorial é a **implementação de referência** da lógica do grafo, não um arquivo a ser digitado.

Este tutorial dá continuidade direta ao Encontro 1 — o `GameManager` já deve existir e estar registrado como Autoload antes de começar.

Objetivos da semana

- Explicar a centralização de input de alto nível no próprio Player como característica arquitetural do Godot.
- Explicar o `SaveManager` (Autoload) como mecanismo de persistência de dados entre cenas.
- Implementar uma variável persistente no `SaveManager` (Orchestration) e validar sua persistência entre trocas de cena.

Resultado esperado ao final da semana

Ao final da Semana 4 (Encontros 1 e 2), cada estudante terá um `GameManager` e um `SaveManager` configurados como Autoload, com pelo menos um dado de progresso persistindo corretamente entre cenas. Este tutorial cobre apenas o **Encontro 2**: a discussão sobre input centralizado e a construção do `SaveManager`.

Pré-requisitos

- `GameManager` criado como Orchestration e registrado como Autoload, com `spawn_player()` e a variável de estado do desafio do Encontro 1 (ver Tutorial - Semana 4,

Encontro 1).

- Uso básico do editor do Orchestrator (grafo, nós, pinos, variáveis, funções).

Antes de começar

O que o estudante deverá possuir antes desta semana

- O projeto do Encontro 1, com o `GameManager` registrado como Autoload e funcionando.

Arquivos necessários

- Nenhum arquivo externo adicional. O `SaveManager` é uma Orchestration nova, criada dentro do próprio projeto.

Assets utilizados

- Nenhum asset novo.

Projeto esperado

- Projeto aberto no Godot 4.7 com o Orchestrator ativado, e o `GameManager` do Encontro 1 já registrado e testado.
- Uma segunda Scene de teste simples, criada neste encontro exclusivamente para validar a persistência de dados entre trocas de cena (ver Parte 3).

Imagem sugerida

Objetivo: mostrar a aba Autoload de Project Settings com dois registros — `GameManager` e `SaveManager` — lado a lado. Enquadramento: captura de tela da janela Project Settings, aba Autoload. Elementos importantes: lista com os dois Autoloads, coluna Path (ambas apontando para arquivos `.torch`) e coluna Node Name. Destaque: as duas linhas, reforçando que são Autoloads independentes. Legenda sugerida: "GameManager e SaveManager registrados como Autoloads independentes ao final da Semana 4."

Parte 1 — Input de alto nível centralizado no Player

Objetivo

Entender uma escolha arquitetural do Godot antes de seguir para a construção do `SaveManager` : por que o próprio Player concentra a leitura de ações de gameplay, sem um objeto de controle separado.

Conceito

A Unreal Engine separa formalmente "quem controla" (PlayerController) de "o que é controlado" (Pawn/Character) — o Controller lê o input e envia comandos ao Pawn que ele está possuindo, permitindo, por exemplo, trocar de Pawn sem perder o Controller. O Godot não possui uma separação nativa equivalente: o próprio Node do Player (o `CharacterBody3D` já configurado desde a Semana 1) concentra tanto a leitura de input de alto nível — ações de gameplay como interagir, pausar ou atacar, não apenas movimentação — quanto a lógica que reage a esse input.

Essa não é uma limitação, mas uma escolha arquitetural válida: projetos pequenos e médios raramente precisam trocar o personagem controlado em tempo de execução, e concentrar o input no próprio Player reduz a quantidade de camadas de indireção necessárias para implementar uma ação simples. É essa mesma centralização que permite, nas próximas semanas, que o Player leia diretamente ações do Input Map (Semana 2) para abrir o menu de pausa ou interagir com um objeto do mundo, sem precisar de um Controller intermediário.

Passo a passo

Esta parte não possui etapas de implementação no editor — é uma discussão conceitual conduzida em aula, sem alteração no projeto.

1. Revisar com a turma como o Input Map e o `_input()` / `_unhandled_input()` do Player já funcionam desde a Semana 2.
2. Explicar a separação Pawn/Controller da Unreal e o papel de cada parte.
3. Confirmar que o Godot resolve o mesmo problema sem essa separação, concentrando a leitura de input de alto nível no próprio Player.
4. Relacionar essa escolha ao que será construído nas próximas semanas: o Player, e não um Controller separado, será responsável por acionar interações (Semana 5) e abrir o menu de pausa (Semana 9).

Resultado esperado

A turma reconhece a ausência de uma separação nativa Pawn/Controller no Godot como uma escolha arquitetural, não uma limitação, e entende que o Player continuará concentrando input de alto nível nas próximas semanas.

Verificando

1. Confirmar, em discussão, que os estudantes conseguem explicar o que o PlayerController resolve na Unreal, sem entrar em detalhes de implementação da engine.
2. Confirmar que os estudantes relacionam corretamente a centralização de input no Player a decisões futuras do Vertical Slice (interação, pausa).

Problemas comuns

- Tentar implementar uma separação manual entre "controller" e "pawn" no projeto, replicando a Unreal sem necessidade: reforçar que essa separação está fora do escopo do Vertical Slice e não traz benefício para um projeto deste porte.
- Tratar a ausência de Pawn/Controller como uma falha do Godot: reforçar que é uma escolha de design, com trade-offs próprios, não uma limitação técnica.

Boas práticas

- Manter essa discussão breve, sem consumir tempo de laboratório destinado ao `SaveManager`.
- Registrar essa comparação como um ponto de análise arquitetural válido para a Semana 17 (ver `PROJECT_ARCHITECTURE.md`, seção 12).

Comparação com Unity

A centralização de input de alto nível no próprio Player, sem uma separação nativa equivalente a Pawn/Controller, aproxima o Godot da forma como a maioria dos projetos em Unity já lida com input de gameplay — tipicamente concentrado no próprio script do personagem ou em um Input System conectado diretamente a ele, sem um objeto de controle formalmente separado. Assim como o Godot, a Unity não impõe uma separação formal Pawn/Controller: ambas deixam essa decisão a cargo do time.

Parte 2 — Criando e registrando o SaveManager

Objetivo

Construir o segundo Autoload do projeto, dedicado exclusivamente a manter dados que precisam sobreviver à troca de cena.

Conceito

Toda engine multi-cena precisa de um lugar para guardar dados que sobrevivem à troca de cena — antes mesmo de qualquer gravação em disco. Esse é o papel do `SaveManager`: um Autoload independente do `GameManager`, dedicado exclusivamente a manter dados persistentes entre cenas e centralizar o slot de save ativo, preparando o terreno para a serialização em disco que será construída na Semana 7 (via `SaveComponent` / `SaveData`).

Separar `SaveManager` de `GameManager` não é redundância: o `GameManager` guarda regras e estado da partida atual (por exemplo, se a porta principal está aberta); o `SaveManager` guarda dados que precisam sobreviver à própria troca de cena, e futuramente à gravação em arquivo. Essa distinção evita que um único Autoload acumule responsabilidades que deveriam estar separadas — princípio já reforçado no Encontro 1 (Parte 1).

A construção segue o mesmo padrão do `GameManager`: uma Orchestration `Node` registrada diretamente como Autoload.

Passo a passo

1. No FileSystem Dock, dentro de `orchestrations/autoload/`, crie uma nova Orchestration chamada `save_manager.torch`, com classe base `Node`.
2. Abra-a no editor do Orchestrator e registre uma nota curta explicando que este Autoload guarda dados que precisam sobreviver a trocas de cena, distinto do `GameManager`.
3. Abra **Project > Project Settings > Autoload**.
4. No campo **Path**, selecione `res://orchestrations/autoload/save_manager.torch`.
5. Confirme que o **Node Name** aparece como `SaveManager` (PascalCase) e clique em **Add**.
6. Confirme que a lista de Autoloads agora mostra `GameManager` e `SaveManager`, ambos habilitados.
7. Salve o projeto (**Ctrl+S**).

Resultado esperado

Existe um segundo Autoload, `SaveManager` (Orchestration `save_manager.torch`), registrado e habilitado em Project Settings, independente do `GameManager` do Encontro 1.

Verificando

1. Abra **Project Settings > Autoload** e confirme as duas linhas: `GameManager` e `SaveManager`, ambas apontando para arquivos `.torch`.
2. Rode qualquer Scene do projeto e confirme, via teste temporário (`print(SaveManager)` / `nó Print`) em qualquer script ou grafo, que o novo Autoload é acessível globalmente.

Problemas comuns

- Registrar o `SaveManager` reaproveitando o mesmo Node Name do `GameManager` por engano: revisar o campo Node Name antes de clicar em Add, garantindo que o nome corresponda exatamente ao Autoload criado.
- Colocar, por atalho, uma variável que deveria estar no `SaveManager` diretamente no `GameManager` (ou vice-versa): revisar a distinção de responsabilidades antes de declarar qualquer variável nova nos próximos passos.

Boas práticas

- Manter `GameManager` e `SaveManager` como Orchestrations totalmente independentes — nenhuma deve depender de detalhes internos da outra.
- Registrar, logo na nota de topo de cada Orchestration de Autoload, qual tipo de dado ela guarda — essa documentação mínima evita duplicação de responsabilidade nas semanas seguintes.

Comparação com Unity

A Unity resolve persistência entre cenas de forma equivalente a um Autoload por meio de um singleton manual — muitas vezes o mesmo objeto usado para o "game manager" do projeto, mantido vivo com `DontDestroyOnLoad`. O Godot separa esse papel em um segundo Autoload dedicado, reforçando a especialização de responsabilidades entre `GameManager` e `SaveManager` de uma forma que, na Unity, depende inteiramente da disciplina da equipe para não se misturar em um único objeto.

Parte 3 — Implementando e testando a persistência entre cenas

Objetivo

Validar, na prática, que uma variável guardada no `SaveManager` sobrevive a uma troca de cena real dentro do projeto.

Conceito

Um Autoload só demonstra sua utilidade quando testado sob o cenário que ele resolve: a troca de cena. Uma variável declarada dentro de `level_exploration.tscn` seria destruída ao carregar outra cena; uma variável declarada no `SaveManager` não. Este passo cria uma segunda Scene de teste mínima, exclusivamente para essa validação — ela não faz parte do nível final do Vertical Slice, que continuará sendo construído a partir de `level_exploration.tscn` nas próximas semanas.

Passo a passo

1. Na Orchestration `save_manager.torch`, no painel de variáveis, declare uma variável persistente simples — implementação de referência:

```
var itens_coletados: int = 0
```

2. Crie uma **função pública** para alterá-la — implementação de referência:

```
func coletar_item() -> void:  
    itens_coletados += 1
```

No grafo: função `coletar_item` → nó **Set** da variável `itens_coletados` recebendo `itens_coletados + 1`.

3. No FileSystem Dock, crie uma nova Scene em `scenes/levels/exploration/` chamada `level_teste_persistencia.tscn`, com um Node raiz `Node3D` e um `Label3D` filho exibindo um texto temporário (por exemplo, "Cena de teste"), seguindo a mesma pasta de `level_exploration.tscn` (PROJECT_ARCHITECTURE.md, seção 8).

4. No Player (`player.tscn`), adicione temporariamente uma ação de teste: ao pressionar uma tecla ainda não usada (por exemplo, `ui_accept`), chame `SaveManager.coletar_item()` e imprima o valor — implementação de referência:

```
func _unhandled_input(event: InputEvent) -> void:
    if event.is_action_pressed("ui_accept"):
        SaveManager.coletar_item()
        print(SaveManager.itens_coletados)
```

No Orchestrator: evento de input → nó **Call Function** `coletar_item` com alvo vindo de **Get Autoload** (`SaveManager`) → nó **Print** com `SaveManager.itens_coletados` .

5. Rode `level_exploration.tscn` (F6), pressione a tecla de teste algumas vezes e confirme, no Output, que o valor de `itens_coletados` aumenta.
6. Ainda no editor, troque manualmente a Scene principal rodada para `level_teste_persistencia.tscn` (Play Current Scene com essa Scene aberta, ou defina-a temporariamente como Main Scene em Project Settings).
7. Em qualquer script/gráfo anexado a um Node dessa Scene de teste, ou diretamente no depurador, confirme que `SaveManager.itens_coletados` mantém o valor acumulado no passo anterior, mesmo após a troca de cena.
8. Reverta a Main Scene do projeto para `level_exploration.tscn` em Project Settings, caso tenha sido alterada no passo 6.
9. Remova a ação de teste temporária do Player, mantendo apenas a variável e a função no `SaveManager` .
10. Salve todos os grafos e cenas alterados (**Ctrl+S**).

Resultado esperado

O `SaveManager` mantém o valor de `itens_coletados` (ou variável equivalente escolhida) mesmo após a troca entre `level_exploration.tscn` e `level_teste_persistencia.tscn` , confirmando que o dado sobrevive à troca de cena.

Verificando

1. Confirme que `itens_coletados` aumenta corretamente ao acionar a função de teste em `level_exploration.tscn` .
2. Troque para `level_teste_persistencia.tscn` e confirme que o valor não é reiniciado para zero.
3. Confirme que a Main Scene do projeto foi revertida para `level_exploration.tscn` ao final do teste.

Problemas comuns

- Declarar a variável de teste dentro da Scene (por exemplo, num grafo anexado a um Node de `level_exploration.tscn`) em vez de no `SaveManager`: o valor será perdido ao trocar de cena — esse é exatamente o erro que a Parte 3 existe para evitar, revisando onde a variável foi declarada.
- Esquecer de reverter a Main Scene do projeto para `level_exploration.tscn` após o teste, deixando o projeto abrindo na Scene de teste por engano.
- Confundir o papel do `SaveManager` (persistência entre cenas, em memória) com gravação em disco: nenhum arquivo é salvo neste encontro — a serialização em disco só será introduzida na Semana 7.
- Esquecer de marcar `coletar_item` como função pública na Orchestration: `SaveManager.coletar_item()` não será reconhecido pelos outros nós/scripts.

Boas práticas

- Sempre testar persistência com uma troca de cena real, nunca apenas revisando o grafo — o comportamento de um Autoload só é confiável quando observado em funcionamento.
- Remover Scenes de teste temporárias (como `level_teste_persistencia.tscn`) da Main Scene do projeto assim que o teste for concluído, evitando que builds futuros abram na cena errada.
- Nomear variáveis do `SaveManager` de forma que já sugiram o que representam no Vertical Slice final (`itens_coletados`, não `contador` ou `x`).

Comparação com Unity

Assim como discutido na Parte 2, a Unity resolveria esse mesmo teste com um singleton mantido por `DontDestroyOnLoad`, validado da mesma forma — trocando de cena via `SceneManager.LoadScene` e conferindo se o valor da variável persiste no objeto singleton. O comportamento esperado é idêntico entre as duas engines; o que muda é que, no Godot, a sobrevivência do Autoload já é garantida pelo registro em Project Settings, sem exigir nenhuma chamada explícita equivalente a `DontDestroyOnLoad` dentro do próprio grafo.

Ao final da semana

Ao final da Semana 4 (Encontros 1 e 2), o projeto do Vertical Slice deve conter:

- O Player, o nível de teste e o build da Semana 3, sem nenhuma alteração.

- Um `GameManager` (Autoload, Orchestration), com `spawn_player()` posicionando o Player no `PlayerStart` e pelo menos uma variável de estado de partida própria (desafio do Encontro 1).
- Um `SaveManager` (Autoload, Orchestration), independente do `GameManager`, com pelo menos uma variável persistindo corretamente entre cenas (Encontro 2).
- A discussão conceitual sobre centralização de input no Player, sem alteração de código associada.

Segundo o `PROJECT_ARCHITECTURE.md` (seção 6, Módulo 2), este resultado corresponde à conclusão do item "GameManager (Autoload)" e ao início do item "Player input de alto nível + SaveManager (Autoload)" do roadmap. Os dois Autoloads construídos nesta semana sustentam toda a arquitetura de gameplay das semanas seguintes — a partir da Semana 5, o contrato `Interactable` e as Signals passarão a se comunicar com o `GameManager` ao reagir a interações do jogador.

Desafio

Cada grupo define e implementa, no `SaveManager`, um dado próprio que deve persistir entre cenas — pontuação, item coletado ou estado de progresso diferente do usado na demonstração —, validando a persistência com uma troca de cena real dentro do projeto, seguindo o mesmo procedimento da Parte 3.

Checklist

- ☐ Discussão sobre centralização de input no Player realizada, sem alteração de código
- ☐ Orchestration `save_manager.torch` criada (classe base `Node`), com nota de responsabilidade registrada
- ☐ `SaveManager` registrado e habilitado em **Project Settings > Autoload**, junto ao `GameManager`, apontando para `save_manager.torch`
- ☐ Variável persistente e função pública implementadas no `SaveManager` e validadas com troca real de cena
- ☐ Main Scene do projeto revertida para `level_exploration.tscn` após o teste
- ☐ Desafio do grupo (dado próprio persistente) implementado e validado

Glossário

- **SaveManager:** Autoload responsável por manter dados persistentes entre cenas e centralizar o slot de save ativo, base para a serialização em disco da Semana 7.

- **Orchestration:** grafo de visual scripting do Orchestrator (arquivo `.torch`); pode ser registrada diretamente como Autoload, como um `.gd`.
- **Implementação de referência:** trecho de GDScript no tutorial que descreve a lógica que o grafo deve reproduzir; não é um arquivo a ser digitado.
- **Get Autoload (Orchestrator):** nó que devolve a referência de um Autoload registrado, para chamar suas funções e ler suas variáveis a partir de outro grafo.
- **Pawn/Controller:** separação nativa da Unreal Engine entre o objeto controlado (Pawn) e quem lê o input e comanda esse objeto (Controller); ausente no Godot, que concentra essa responsabilidade no próprio Node do Player.
- **Persistência entre cenas:** capacidade de um dado sobreviver à troca de Scene ativa, sem depender ainda de gravação em disco.

Referências

- Godot Documentation — Singletons (Autoload):
https://docs.godotengine.org/en/stable/tutorials/scripting/singletons_autoload.html
- Godot Documentation — GDScript:
<https://docs.godotengine.org/en/stable/tutorials/scripting/gdscript/index.html>
- Orchestrator — Documentação oficial: <https://orchestrator.cratercrash.space/>
- Orchestrator — Autoloads: <https://docs.cratercrash.space/orchestrator/nodes/autoloads/>
- Unity Manual (consulta comparativa) — DontDestroyOnLoad:
<https://docs.unity3d.com/ScriptReference/Object.DontDestroyOnLoad.html>
- Canais recomendados para consulta complementar (não substituem a documentação oficial):
GDQuest (<https://www.youtube.com/@GDQuest>), Clear Code
(<https://www.youtube.com/@ClearCode>)